

Лабораторная работа №4

Тема: «Контейнерные классы стандартной библиотеки языка. Класс Вектор»

Цель работы: Знакомство с контейнерными классами в C++ . Изучение векторов в языке C++ . Знакомство с методами класса, Итераторами и функциями стандартной библиотеки STL для работы с контейнерами.

Векторы

Вектор -- это структура данных, которая хранит элементы, подобно обычному массиву. Главное отличие вектора от массива C++ состоит в том, что вектор всегда "знает", сколько элементов он хранит, и может динамически изменять свой размер. Добавление новых элементов в конец вектора выполняется очень быстро, но операция по вставке новых элементов в начало или в середину вектора требует значительных затрат по времени.

В STL, класс вектора носит имя `std::vector<T>` и определен в заголовке `<vector>`. Объявить вектор, который будет хранить массив экземпляров класса `Film`, можно так:

```
vector<Film> films;
```

Вектор, созданный подобным образом, изначально имеет размер 0. Если заранее известно количество элементов в векторе, можно явно указать начальный размер в определении и с помощью оператора `"[ ]"` присвоить значения его элементам.

Очень удобно заполнять вектор с помощью функции `push_back()`. Она добавляет указанный элемент в конец вектора, увеличивая его размер на 1:

```
films.push_back(Film(4812, "A Hard Day's Night",  
85));  
films.push_back(Film(5051, "Seven Days to Noon",  
94));  
films.push_back(Film(1301, "Day of Wrath",  
105));  
films.push_back(Film(9227, "A Special Day",  
110));  
films.push_back(Film(1817, "Day for Night",  
116));
```

Еще один способ заполнения вектора состоит в том, чтобы задать при объявлении его начальный размер, а потом выполнить инициализацию отдельных элементов:

```
vector<Film> films(5);  
  
films[0] = Film(4812, "A Hard Day's Night", 85);  
films[3] = Film(9227, "A Special Day", 110);  
films[4] = Film(1817, "Day for Night", 116);
```

Элементы вектора, которые не были инициализированы явно, приобретают значения, присвоенные конструктором по умолчанию. В случае базовых типов языка C++ и указателей, начальные значения элементов вектора не определены, аналогично локальным переменным, размещаемым на стеке.

Векторы допускают обход всех элементов в цикле, с использованием оператора `[]`:

```
for (int i = 0; i < (int)films.size(); ++i)
    cerr << films[i].title().ascii() << endl;
```

В качестве альтернативы -- можно использовать итератор:

```
vector<Film>::const_iterator it = films.begin();

while (it != films.end()) {
    cerr << (*it).title().ascii() << endl;
    ++it;
}
```

Каждый контейнерный класс имеет два типа итераторов: `iterator` и `const_iterator`. Различие между ними заключается в том, что `const_iterator` не позволяет модифицировать элементы вектора.

Функция-член контейнера -- `begin()` возвращает итератор, который ссылается на первый элемент в контейнере (например, `films[0]`).

Функция-член контейнера -- `end()` возвращает итератор, который ссылается на элемент "следующий за последним" (например, `films[5]`).

Если контейнер пуст, значения, возвращаемые функциями `begin()` и `end()`, эквивалентны. Это обстоятельство может использоваться для проверки наличия элементов в контейнере, хотя для этой цели гораздо удобнее использовать функцию `empty()`.

Итераторы обладают интуитивно понятным синтаксисом, который напоминает синтаксис указателей языка C++. Для перемещения к следующему или предыдущему элементу, можно использовать

операторы "++" и "--", и унарный "*" -- для получения доступа к элементу контейнера, находящемуся в позиции итератора.

Если необходимо отыскать некоторый элемент в векторе, можно воспользоваться функцией STL -- `find()`:

```
vector<Film>::iterator it =  
find(films.begin(), films.end(),  
Film(4812));  
if (it != films.end())  
    films.erase(it);
```

Она возвращает итератор, указывающий на первый встретившийся элемент вектора, отвечающий критериям поиска (элементы контейнера сравниваются перегруженным `operator==()` с последним аргументом функции). Определение функции находится в заголовке `<algorithm>`, где вы найдете множество других шаблонных функций

Сортировка элементов вектора может быть произведена функцией `sort()`:

```
sort(films.begin(), films.end());
```

Для сравнения элементов вектора она использует оператор "<", если явно не указывается другая функция сравнения. На отсортированных векторах, для поиска некоторого элемента может использоваться функция `binary_search()`. Она дает результат, аналогичный `find()` (при условии, что в векторе нет двух фильмов с одинаковыми числовыми идентификаторами), но при этом работает намного быстрее.

```
int id = 1817;  
if (binary_search(films.begin(),  
films.end(), Film(id)))  
    cerr << "Found " << id << endl;
```

В позицию итератора, с помощью функции `insert()`, может быть вставлен новый элемент или удален существующий, с помощью функции `erase()`:

```
films.erase(it);
```

Элементы, которые следуют за удаляемым будут перемещены на одну позицию влево (или выше, если хотите) и размер вектора будет уменьшен на 1 элемент.

Задание. Составить алгоритм решения задачи согласно своему варианту и написать программу на языке C++, используя **Класс vector**. В программе **МАКСИМАЛЬНО использовать стандартную библиотеку**.

В отчете предоставить перечень использованных методов класса и функций стандартной библиотеки и текст программы со скриншотами результатов ее работы.

Варианты заданий

Создать вектор из объектов класса, разработанного вами в лабораторной работе №1 этого семестра.

Создать новый вектор, переписав в него некоторые объекты исходного вектора (критерий для перезаписи выберите самостоятельно)

Проверить, не оказался ли новый вектор пустым.

Если не пуст, провести сортировку объектов в новом векторе. (критерий сортировки тоже выберите сами, исходя из атрибутов объектов вашего класса)

Реализуйте возможность вставки в новый вектор нового объекта, не нарушающего условие сортировки.

Примечание. Размерности векторов задаются с клавиатуры.